SIGOPS SDK

Developer Toolkit for Tools, Templates, Connectors & Plugins

6 NPM Packages | CLI | SEL Tooling | VS Code Extension | MIT License

Version v1.0.0 | April 2026

MIT License (Open Source)

Table of Contents

1. Executive Summary

The SigOps SDK is the developer toolkit for building tools, templates, connectors, and plugins for the SigOps ecosystem. It enables third-party developers to extend SigOps with custom automation capabilities and sell them on the SigOps Marketplace.

The SDK is fully open source (MIT license) and published as 6 NPM packages under the @sigops scope. It includes a CLI for scaffolding, testing, validating, and publishing; language tooling for the SigOps Execution Language (SEL); and development libraries for tools, templates, and plugins.

Metric	Value
Repo	sigops-sdk (PUBLIC, MIT License)
Packages	6 NPM packages under @sigops/*
Targets	Tool developers, DevOps engineers, MSPs, SaaS vendors
Language	TypeScript (Node.js 20+)
Marketplace	SigOps Cloud — 75% creator / 25% SigOps revenue share
Dependencies	sigops core (SEL parser shared), Zod (validation)

2. Package Architecture

2.1 Package Overview

Package	Purpose	Size	Depends On
@sigops/cli	Developer CLI: scaffold, test, validate, publish, SEL lint/format	~2MB	tool-sdk, template-sdk, sel-tools
@sigops/tool-sdk	Library for creating custom tools with typed I/O	~50KB	zod
@sigops/template-sdk	Library for creating parameterized .sel templates	~40KB	sel-tools, zod
@sigops/sel-tools	SEL parser, linter, formatter, Language Server Protocol	~200KB	(extracted from sigops core)
@sigops/vscode-ext	VS Code extension: syntax, autocomplete, diagnostics, hover	~1MB	sel-tools
@sigops/plugin-sdk	Library for creating engine plugins with hook system	~60KB	zod

2.2 Dependency Graph

The packages form a clean dependency tree with no circular dependencies:

@sigops/cli → @sigops/tool-sdk → zod

@sigops/cli → @sigops/template-sdk → @sigops/sel-tools → (SEL parser from sigops core)

@sigops/cli → @sigops/sel-tools

@sigops/vscode-ext → @sigops/sel-tools

@sigops/plugin-sdk → zod (standalone, hooks into sigops engine at runtime)

The SEL parser is extracted from sigops core and published as part of @sigops/sel-tools. Both sigops core and the SDK consume the same parser — ensuring .sel files are always parsed identically.

2.3 Repository Structure

```
sigops-sdk/
  packages/
    cli/              @sigops/cli
    tool-sdk/         @sigops/tool-sdk
    template-sdk/     @sigops/template-sdk
    sel-tools/        @sigops/sel-tools
    vscode-ext/       @sigops/vscode-ext
    plugin-sdk/       @sigops/plugin-sdk
  examples/
    tool-restart-service/
    tool-slack-notify/
    tool-db-backup/
    tool-http-health-check/
    template-cpu-fix/
    template-disk-cleanup/
    template-deploy-canary/
    plugin-custom-metric/
  docs/
    getting-started.md
    tool-reference.md
    template-guide.md
    sel-reference.md
    plugin-guide.md
    publishing.md
  pnpm-workspace.yaml
  turbo.json
```

3. Four Sellable Item Types

The SigOps Marketplace supports four types of sellable items. The SDK provides development libraries and CLI support for all four:

Type	What It Is	SDK Package	Example
Tool	A reusable action (restart, backup, notify, query)	@sigops/tool-sdk	Database backup to S3
Template	A parameterized .sel workflow with fill-in-the-blank parameters	@sigops/template-sdk	CPU alert auto-fix template
Connector	An integration to an external system (DB, API, SaaS)	@sigops/tool-sdk (extended)	Zendesk ticket connector
Plugin	An engine extension that hooks into execution lifecycle	@sigops/plugin-sdk	Custom metric collector

YES — the SDK fully supports template development. @sigops/template-sdk provides defineTemplate(), ParameterSchema, TemplateRenderer, and TestRunner. Templates are parameterized .sel files that generate complete workflows from user-provided parameters.

4. Tool System Architecture

4.1 Tool Interface

Every tool implements the ToolDefinition interface. The tool-sdk provides a defineTool() helper that enforces this interface with full TypeScript type safety:

Property	Type	Required	Description
name	string	Yes	Unique name: org.tool-name (e.g., acme.db-backup)
version	string	Yes	Semver (e.g., 1.0.0)
description	string	Yes	Human-readable description
input	ZodSchema	Yes	Zod schema for input validation
output	ZodSchema	Yes	Zod schema for output shape
execute	(input, ctx) => Promise<output>	Yes	The tool logic
rollback	(input, ctx) => Promise<void>	No	Optional rollback logic
healthCheck	(ctx) => Promise<boolean>	No	Optional connectivity check
timeout	number	No	Max execution time (ms, default 30000)
retryable	boolean	No	Whether engine can retry on failure

4.2 ToolContext

Every tool receives a ToolContext object providing access to execution environment:

Property	Type	Description
logger	Logger	Structured logger (info, warn, error, debug)
secrets	SecretStore	Access tenant secrets: secrets.get('DB_PASSWORD')
agent	AgentInfo	Agent hostname, OS, IP, labels, available tools
tenant	TenantInfo	Tenant ID, name, plan
signal	Signal | null	The triggering signal (null for manual/scheduled)
execution	ExecutionInfo	Execution ID, workflow ID, step index
abort	AbortSignal	Cancellation signal from engine
metrics	MetricsEmitter	Emit custom metrics: metrics.gauge('backup_size', bytes)

4.3 Tool Lifecycle

1. Developer creates tool using defineTool() with Zod schemas for input/output.
2. Developer tests locally using sigops test (mock ToolContext, no engine needed).
3. Developer validates using sigops validate (schema check, security scan, test coverage).
4. Developer publishes using sigops publish (uploads to SigOps Cloud marketplace).
5. User installs tool from marketplace into their SigOps instance.
6. Engine discovers tool and registers it in the tool registry.
7. SEL workflows reference the tool by name: tool: 'acme.db-backup'.
8. At execution time, engine validates input against Zod schema, runs execute(), validates output.
9. If tool fails and retryable=true, engine retries with exponential backoff.
10. If rollback is defined and execution fails, engine calls rollback().

5. Template System Architecture

5.1 What Templates Are

Templates are parameterized .sel workflow blueprints. A template author defines parameters (using Zod schemas) and a render function that generates valid SEL code from those parameters. Users fill in parameters via the UI or CLI to generate a complete, ready-to-run workflow.

Templates solve the cold-start problem: new SigOps users get production-ready workflows in minutes instead of writing SEL from scratch. The marketplace enables community-driven templates for every infrastructure pattern.

5.2 Template Interface

Property	Type	Required	Description
name	string	Yes	Unique name: org.template-name
version	string	Yes	Semver
description	string	Yes	What this template automates
category	string	Yes	infrastructure/deployment/monitoring/ security/database/networking
tags	string[]	No	Searchable tags
parameters	ZodSchema	Yes	Zod schema for template parameters
render	(params) => string	Yes	Generates .sel code from parameters
validate	(params) => ValidationResult	No	Custom parameter validation beyond Zod
preview	(params) => string	No	Generate human-readable preview
requiredTools	string[]	No	Tools that must be installed
requiredConnectors	string[]	No	Connectors that must be configured

5.3 Template Development Flow

1. sigops init template my-cpu-fix — scaffolds template project with boilerplate.
2. Define parameters using Zod: z.object({ service: z.string(), threshold: z.number().default(90) }).
3. Write render() function that generates .sel code with parameters interpolated.
4. sigops test --params '{"service":"nginx","threshold":85}' — renders and validates .sel output.
5. The rendered .sel is parsed by the SEL parser to catch syntax errors at authoring time.
6. sigops validate — checks parameter schema, render output validity, example coverage.
7. sigops publish — uploads to marketplace under 'Templates' category.
8. Users browse templates, fill parameters via UI form (auto-generated from Zod schema), click 'Create Workflow'.

5.4 Template Composition

Templates can compose: a 'Full Stack Monitoring' suite template can reference child templates ('CPU Alert Fix', 'Disk Cleanup', 'Memory Alert') and generate a combined workflow. The parent template's render() calls child renders and concatenates the .sel output.

5.5 Playbook-to-Template Conversion

Existing .sel playbooks can be converted to templates with one command: sigops template from-playbook playbook.sel. The CLI analyzes the .sel file, identifies hardcoded values (service names, thresholds, channel names), and generates a template with those values as parameters. The developer reviews, adjusts parameter types and defaults, and publishes.

6. SEL Language Tooling

6.1 SEL Parser

The SEL parser is shared between sigops core and the SDK. It parses .sel source code into an Abstract Syntax Tree (AST). The parser supports: signal matching blocks, step definitions, conditional branches (when/otherwise), retry policies, rollback blocks, escalation rules, variable interpolation, parallel/sequential execution, and comments.

6.2 SEL Linter

Static analysis rules that catch common mistakes and security issues:

Rule	Severity	Description
no-infinite-retry	error	Retry without max_attempts or backoff
require-verify-step	warning	Workflow modifies state but has no verification step
max-parallel-steps	warning	More than 10 parallel steps (resource exhaustion risk)
secret-in-plain-text	error	Hardcoded passwords, API keys, or tokens in .sel
unreachable-branch	warning	Condition that can never be true
missing-rollback	info	Destructive action without rollback defined
undefined-tool	error	References a tool not in the registry
unused-variable	warning	Variable declared but never referenced
timeout-too-long	warning	Step timeout exceeds 5 minutes
no-escalation	info	Critical workflow without escalation path

6.3 SEL Language Server (LSP)

Full Language Server Protocol implementation for IDE integration. Features: real-time error highlighting as you type, autocomplete for tool names (from registry), signal field names, SEL keywords. Hover documentation showing tool descriptions, input/output schemas. Go-to-definition for tool and workflow references. Signature help showing parameter types. Code actions for quick fixes (add missing rollback, fix indentation).

6.4 VS Code Extension

Published to VS Code Marketplace (free). Features: .sel syntax highlighting via TextMate grammar. Full LSP integration (autocomplete, errors, hover, go-to-def). SEL Playground panel (edit + preview AST + test against mock signals). 'Run in SigOps' command (sends workflow to connected SigOps instance). Scaffold tool/template via command palette. Format document (sigops sel fmt).

7. Plugin System Architecture

7.1 Hook Points

Plugins hook into the SigOps engine execution lifecycle at defined points:

Hook Point	When It Fires	Use Case
AFTER_SIGNAL_INGEST	After signal is normalized and stored	Enrich signals with custom data
BEFORE_RULE_MATCH	Before signal matching rules evaluate	Add dynamic rules
BEFORE_EXECUTION	Before workflow execution starts	Pre-flight checks, approvals
AFTER_STEP	After each step completes	Custom metrics, logging
AFTER_EXECUTION	After workflow completes or fails	Custom reporting, webhooks
BEFORE_ROLLBACK	Before rollback is triggered	Custom rollback logic
ON_AGENT_CONNECT	When an agent connects via WebSocket	Agent validation, enrichment
ON_SCHEDULE_TICK	When scheduler evaluates cron expressions	Custom scheduling logic

7.2 UI Extension Points

Plugins can register custom UI panels in the SigOps dashboard: dashboard.sidebar (left sidebar panel), dashboard.main (main content area tab), signal.detail (panel in signal detail view), execution.detail (panel in execution detail view), settings.tab (custom settings tab). Panels are React components bundled with the plugin.

8. Publishing Pipeline

8.1 Validation Steps

Before publishing, sigops validate runs these checks:

Check	What It Does	Blocks Publish?
Schema validation	Zod schemas parse correctly, input/output well-typed	Yes
Type check	TypeScript compiles without errors	Yes
Test coverage	Unit tests exist and pass, coverage > 80%	Yes
Security scan	No hardcoded secrets, no eval(), no network calls in validate	Yes
Permissions audit	Declared permissions match actual usage	Yes
Sandbox run	Execute in isolated environment with mock context	Yes
Size check	Package < 5MB compressed	Yes
Dependency scan	No known vulnerabilities in dependencies	Warning
SEL validation	For templates: rendered .sel parses correctly for all example params	Yes
Documentation	README.md exists with usage instructions	Warning

8.2 Review Process

Private items (within org): instant publish, no review. Public free items: 24-48 hour automated review (extended validation + AI check for quality). Public paid items: 3-5 day manual review by SigOps team (security audit, quality check, pricing review).

8.3 Revenue Share

Creator receives 75% of revenue. SigOps keeps 25%. Payouts monthly via Stripe. Creator dashboard shows installs, revenue, ratings, version analytics. Pricing models: free, one-time purchase, monthly subscription, per-execution, tiered (free tier + paid features).

9. CLI Reference

Command	Description
sigops init tool <name>	Scaffold new tool project
sigops init template <name>	Scaffold new template project
sigops init plugin <name>	Scaffold new plugin project
sigops init connector <name>	Scaffold new connector project
sigops test	Run tool/template locally with mock context
sigops test --input '{...}'	Test with specific input JSON
sigops test --params '{...}'	Test template with specific parameters
sigops validate	Run full validation pipeline
sigops publish	Publish to SigOps Marketplace
sigops publish --private	Publish to org-only (no marketplace)
sigops sel lint <file.sel>	Lint a .sel file
sigops sel fmt <file.sel>	Format a .sel file
sigops sel parse <file.sel>	Parse .sel and print AST
sigops sel playground	Open SEL playground in browser
sigops login	Authenticate with SigOps Cloud API key
sigops whoami	Show current authenticated identity
sigops versions	List published versions of current package
sigops unpublish <version>	Remove a version from marketplace

10. Security Model

Sandboxed execution: tools run in isolated contexts. No filesystem access beyond working directory. No network access unless explicitly declared in permissions. Secrets accessed only via ctx.secrets.get() (never in environment). Timeout enforced by engine (default 30s, max 5min). Memory limit enforced (256MB default). No eval(), no dynamic import(), no child_process. All published packages are signed and integrity-verified.

11. Build Order

Phase	What	When
1	Extract SEL parser from sigops core into @sigops/sel-tools	During Set 1 (with sigops core)
2	Build @sigops/tool-sdk (defineTool + TestHarness + types)	Set 1 (enables early adopters)
3	Build @sigops/cli (init tool + test + validate)	Set 1
4	Build @sigops/template-sdk (defineTemplate + renderer)	Set 2 (Month 2)
5	Extend @sigops/cli (init template + publish + sel lint/fmt)	Set 2
6	Build @sigops/sel-tools (linter rules + formatter + LSP)	Set 2
7	Build @sigops/vscode-ext	Set 2 (Month 3)
8	Build @sigops/plugin-sdk (after sigops-cloud hook system)	Set 2 (Month 4)
9	Write examples/ (5+ example tools + 3+ templates)	Continuous
10	Write docs/ (getting-started, references, guides)	Continuous